

- ① REGEX that accepts strings containing only "abc", repeated at least 3 times, regardless lower/uppercase.

✓ $\text{group} = \text{one abc sequence}$
 $\text{start of string} \leftarrow \text{^} ([aA][bB][cC]) \{3, \} \$$
 $\text{a or A} \quad \text{repeat group at least 3 times} \quad \text{end of string}$

- ② GREP command displaying the lines of ODD length that start and end with a vowel.

✓ $\text{grep -Ei '^[aeiou]([a-z])^* [aeiou]$' filename}$
 $\text{extended regex} \quad \text{ignore case} \quad \text{first character is a vowel} \quad \text{pairs of letters} \quad \text{0 or more} \quad \text{keeps total length odd?}$

- ③ text-file ; at least 4 words ; separated by SPACES on each line (uppercase/lowercase) 2 SED cmds that swap the second word of the line with the penultimate word.

1) $\text{sed -E 's/^([A-Za-z]^+)([A-Za-z]^+)(.*)([A-Za-z]^+)$/\1\4\3\2/' file}$
 $\text{extended regex (newer)} \quad \text{word1 (group1)} \quad \text{word2 (group2)} \quad \text{middle words (group3)} \quad \text{(group4)} \quad \text{3 2 1}$

2) $\text{sed -E 's/^([A-Za-z]^+)([A-Za-z]^+)(.*)([A-Za-z]^+)$/\1\4\3\2/' file}$
 $\text{extended regex (older)} \quad \text{cleaner: } \text{sed -E 's/^([A-Za-z]^+)([A-Za-z]^+)(.*)([A-Za-z]^+)([A-Za-z]^+)$/\1\4\3\2\5/' file}$

- ④ AWK cmd displaying the product of the second and penultimate field of each line if the nr. of fields on the line is ≥ 3 .

1) $\text{awk 'BEGIN {s=0} NF >= 3 {s+=$2} END {print s}' file$
 product

2) $\text{awk 'NF >= 3 {print \$2 * \$ (NF-1)}' filename}$
 $\text{nr. fields } \geq 3 \quad \text{second field} \quad \text{penultimate field}$

⑤ end line in which program B is executed $\Rightarrow$ program A has executed successfully (exit code = 0 $\Rightarrow$ SUCCESS)

✓ A && B

↓
execute second command ($\Rightarrow$) first command (A) exits with code 0.
(B)

ex: gcc a.c -o a && ./a
↓
logic gate

⑥ shell script that reads from a.txt pairs of file names and deletes the second file if first already exists.

#!/bin/bash read pairs
while read file1 file2; do check if first file exists
if [-e "\$file1"]; then
rm "\$file2" remove second file
fi
done < a.txt
reads from a.txt input

⑦ How many child processes will be created by the program below? 0.

int main() {
while (wait(NULL) > 0) {
if (fork() == 0) {
exit(0);
}
}
return 0;
}
no children at this time, so waits for nothing $\Rightarrow$ -1.
 $\Rightarrow$ -1 is not > 0 $\Rightarrow$ while condition is false.
 $\Rightarrow$ never entered the while
 $\Rightarrow$ fork() is never called
 $\Rightarrow$ no children.

⑧ EXEC call that executes the program and arguments given to program ./a :

argv[0] argv[1]
./a ls -l /etc/passwd argv+1
int main(int argc, char** argv) {
...
}

Answer: execvp(argv[1], argv+1)

search path + execute program
ls ls -l /etc/passwd

filename of the program to run

$\rightarrow$ argv array: argv[0] = "./a", argv[1] = "ls", argv[2] = "-l", argv[3] = "/etc/passwd", argv[4] = NULL

- ① send the string "Hello\nWorld\n" to the "grep -E 0" command using popen in write mode. ("w")

```
File *f = popen("grep -E 0", "w");
fprintf(f, "Hello\nWorld\n"); // send string to grep
pclose(f); // close pipe and wait.
```

← program writes to command stdin

- ⑩ What happens if a process tries to write to a full pipe whose write end has been closed by all other processes.

✓ full pipe + reader exists ⇒ block write.
full pipe + no reader ⇒ SIGPIPE / EPIPE

- ⑪ Duration of the program as a function of the abc file type and its permissions.
considering

```
int main() {
    int f;
    char c;
    f = open("abc", O_RDONLY);
    if (f >= 0) {
        read(f, &c, 1);
        close(f);
    }
    return 0;
}
```

- if abc ~~is~~ or abc has no permissions ⇒ open fails quickly. ✓✓
- if abc is a regular, readable file ⇒ read is quick, maybe ~~et~~ cached. ✓
- if abc is FIFO/device/terminal ⇒ read may block while waiting for data ✓
- if abc is directory ⇒ open succeeds, read usually fails ✓

- ⑫ protect critical section:

```
✓ int m = 0;
void * f(void * v) {
    if (m < 500) {
        m++;
    }
    return NULL;
}
```

```
int main() {
    pthread_t t;
    pthread_create(&t, NULL, f, NULL);
    pthread_join(t, NULL);
    return 0;
}
```

→

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER
int m = 0;
void ... {
    pthread_mutex_lock(&m);
    if (m < 500) {
        m++;
    }
    pthread_mutex_unlock(&m);
    return NULL;
}
int main() {
    ...
    pthread_mutex_destroy(&m);
}
```


⑬ Ensure that ≤ 3 persons vote simultaneously.

#include <semaphore.h>

sem_t sem;

void* person(void* args){

sem_wait(&sem); $\rightarrow$ counter decreases (allows to enter)

printf("voting...\n");

sem_post(&sem); $\rightarrow$ counter increases (thread has left)

return NULL;

}

int main(){

pthread_t t[100];

sem_init(&sem, 0, 3);

for(int i=0; i<100; i++){

pthread_create(&t[i], NULL, person, NULL);

}

for(int i=0; i<100; i++){

pthread_join(t[i], NULL);

}

sem_destroy(&sem);

return 0;

}

1 = separate processes

0 = same process

max 3

threads can access the resource simultaneously before locking; count=3 initially

(when all 3 threads are entered/inside,

the count is 0 (counter decreases when

a thread enters) $\Rightarrow$ locked semaphore

inside thread leaves $\Rightarrow$ counter

increases $\Rightarrow$ a 4th thread can

enter now

⑭ What does pthread_cond_wait do with the mutex it gets as an argument?

! When pthread_cond_wait is called, the ^(thread) process ^(unlock) releases the associated mutex and blocks itself; it remains ^(sleep) blocked until another process ^(wake) signals the condition with pthread_cond_signal; after condition is signaled, process is ^(thread) awakened, it acquires the mutex, gets back to running state.

$\rightarrow$ pthread_cond_wait unlocks the mutex and puts the thread to sleep; when signaled, it wakes up the thread and locks the mutex before returning.

⑮ replace all mutexes $\leftrightarrow$ rwlocks

i. all mutex_lock $\leftrightarrow$ rwlock_wrlock

only one thread enters

only one writer enters

i. pthread_mutex_t m $\leftrightarrow$ pthread_rwlock_t rw

$\Rightarrow$ the synchronisation obj. changes.

ii. pthread_mutex_lock(&m) $\leftrightarrow$ pthread_rwlock_wrlock(&rw)

$\Rightarrow$ exclusive (both) meaning only one thread can enter at a time.

Conclusion: should be the same, but rwlocks do more work (allow > readers) ①

⑥ $T, N_1, N_2, N_3 > 0$ s.t. program is OK.

#include <pthread.h>

pthread_barrier_t b1, b2, b3; → declare 3 barriers

void * f1(void * a) { pthread_barrier_wait(&b1); return NULL; } → type-1 threads wait at b1.

... f2 ... → type 2 threads wait at b2

... f3 ... → type 3 threads wait at b3

int main() {

int T, N1, N2, N3;

pthread_t t[T][3]; → T rows, 3 threads per row

pthread_barrier_init(&b1, NULL, N1); // b1 opens when N1 threads arrive

for (int i=0; i<T; i++) { → repeat T times

pthread_create(&t[i][0], NULL, f1, NULL); → create one f1 thread

}

for (int i=0; i<T; i++) { ← wait for all threads

pthread_join(t[i][0], NULL); ← wait for f1

}

pthread_barrier_destroy(&b1); ← destroy b1

return 0;

}

b1 appears 3 times → T calls

b2 → T calls

b3 → T calls

N_i must divide T ⇒ ex: T=6,

$N_1=1$ ✓

$N_2=2$

$N_3=3$

T=5 ⇒ b1 gets T arrivals

$N_1=5$

b2 gets T-1

$N_2=5$

b3 gets 5 T-1

$N_3=5$

2) b1 gets

2) Barrier releases groups of $N_1/N_2/N_3$

⇒ finish ⇒ $T \% N_1 = 0$

$T \% N_2 = 0$

$T \% N_3 = 0$

≠ values ⇒ T=12, $N_1=2$, $N_2=3$, $N_3=4$

⑦ prevent deadlocks caused by locking some mutexes:

- consistent locking order

- avoid nested locks

- keep critical sections short

? - use trylocks if needed.

⑧ SJF vs STCF scheduling policies.

✓ SJF (non-preemptive = cooperative)

• shortest job first (smallest TOTAL runtime)

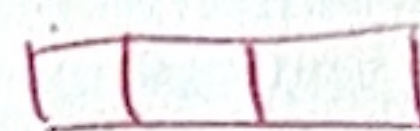
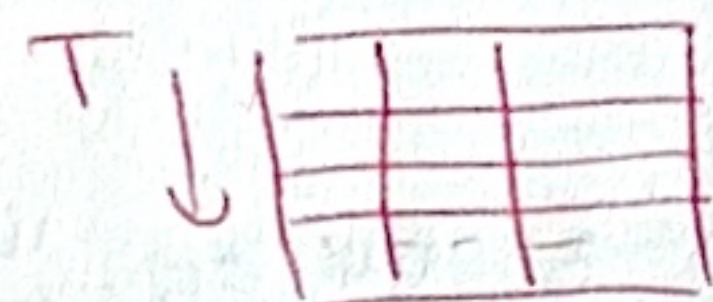
• does not interrupt a running job

STCF (preemptive)

• shortest-time-to-completion first

• can interrupt running job.

(smallest remaining runtime)



(19) A process has unallocated memory zones: 32, 44, 98.

What will the list be after two `malloc(33)` requests, by Worst Fit algorithm?

Worst Fit chooses largest free block.

✓ $32, 44, \underline{98} \xrightarrow{\text{malloc}(33)}$ chosen block: 98
remaining: $98 - 33 = 65$

$\Rightarrow 32, 44, \underline{65} \xrightarrow{\text{malloc}(33)}$ chosen block: 65
remaining: $65 - 33 = 32$

$\Rightarrow 33, 44, 32$

(20) hard vs symbolic link

✓ see (22 jun 2023)